

# DATA STRUCTURES AND ALGORITHMS

Hemavarshini S

## Exercise 2: E-commerce Platform Search Function

Scenario:

You are working on the search functionality of an e-commerce platform. The search needs to be optimized for fast performance.

CODE:

```
import java.util.Arrays;
import java.util.Comparator;

public class Product {
    private int productId;
    private String productName;
    private String category;

    public Product(int productId, String productName, String category) {
        this.productId = productId;
        this.productName = productName;
        this.category = category;
    }

    public int getProductId() { return productId; }
    public String getProductName() { return productName; }
    public String getCategory() { return category; }

    @Override
    public String toString() {
        return "Product{" + "id=" + productId + ", name=" + productName + ", cat=" + category + "}";
    }

    public static Product linearSearch(Product[] products, int targetId) {
        for (Product product : products) {
            if (product.getProductId() == targetId) {
                return product;
            }
        }
        return null;
    }

    public static Product binarySearch(Product[] sortedProducts, int targetId) {
        int left = 0;
        int right = sortedProducts.length - 1;

        while (left <= right) {
```

```

        int mid = left + (right - left) / 2;
        int midId = sortedProducts[mid].getProductId();

        if (midId == targetId) {
            return sortedProducts[mid];
        }
        if (midId < targetId) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return null;
}

public static void main(String[] args) {
    Product[] inventory = {
        new Product(105, "Wireless Mouse", "Electronics"),
        new Product(101, "Running Shoes", "Footwear"),
        new Product(104, "Coffee Mug", "Kitchen"),
        new Product(102, "Leather Wallet", "Accessories")
    };

    System.out.println("--- Linear Search ---");
    Product res1 = linearSearch(inventory, 104);
    System.out.println("Found: " + res1);

    Product[] sortedInventory = Arrays.copyOf(inventory, inventory.length);
    Arrays.sort(sortedInventory, Comparator.comparingInt(Product::getProductId));

    System.out.println("\n--- Binary Search ---");
    Product res2 = binarySearch(sortedInventory, 104);
    System.out.println("Found: " + res2);
}
}

```

#### Output

```

--- Linear Search ---
Found: Product{id=104, name='Coffee Mug', cat='Kitchen'}

--- Binary Search ---
Found: Product{id=104, name='Coffee Mug', cat='Kitchen'}

=== Code Execution Successful ===

```

## Exercise 7: Financial Forecasting

Scenario:

You are developing a financial forecasting tool that predicts future values based on past data.

CODE:

```
public class FinancialForecasting {

    public static double calculateFutureValue(double presentValue, double growthRate, int periods) {

        if (periods <= 0) {
            return presentValue;
        }

        return calculateFutureValue(presentValue, growthRate, periods - 1) * (1 + growthRate);
    }

    public static void main(String[] args) {
        double initialInvestment = 1000.0;
        double annualGrowthRate = 0.05;
        int years = 4;

        System.out.println("--- Financial Forecasting Simulation ---");
        System.out.printf("Initial Principal: $%.2f%n", initialInvestment);
        System.out.printf("Expected Growth Rate: %.1f%%\n", (annualGrowthRate * 100));
        System.out.printf("Forecast Horizon: %d years\n", years);

        double forecastedValue = calculateFutureValue(initialInvestment, annualGrowthRate, years);

        System.out.println("-----");
        System.out.printf("Forecasted Future Value: $%.2f%n", forecastedValue);
    }
}
```

### Output

```
--- Financial Forecasting Simulation ---
Initial Principal: $1000.00
Expected Growth Rate: 5.0%
Forecast Horizon: 4 years
-----
Forecasted Future Value: $1215.51

=== Code Execution Successful ===
```